How much junk do rules catch?
pygarble on real survey responses and social-media text

Ujjwal Rao
[TODO: affiliation]
[TODO: email]
`https://github.com/brightertiger/pygarble`

October 2026

Abstract

Free-text fields collect junk: keyboard mashing, random strings and placeholder answers. pygarble is a Python library with no runtime dependencies that flags such text with rules and small embedded English statistics. We measure what it catches on two public datasets, with every method at its own default decision and no tuning on the test data. On 3,513 survey responses labelled as junk, the `english_fusion` profile flags 0.895. On 9,971 English rows of random strings and tweets, it has precision 0.906, recall 0.991 and a false-positive rate of 6.8 % on tweets. It is not the best on every axis. A Markov-chain baseline flags only 0.3 % of tweets but catches 0.722 of survey junk; a language identifier catches 0.989 of survey junk but flags 9.4 % of tweets and takes 307.47 MB installed. pygarble's default `english` profile flags 33.2 % of tweets. We report every loss, the cost of each method, and which profile to use for which job.

1 Introduction

Text pipelines receive junk. Survey free-text questions get keyboard mashing from respondents who want to reach the next page. Sign-up forms get random strings from bots. Scraped corpora carry encoding damage and boilerplate. Each of these costs something downstream: a coder reads it, a model is trained on it, or an analysis counts it as an answer.

The usual remedies are heavy. A language model or a trained classifier needs a model download, a GPU or an API call, and its decisions are hard to explain row by row. Language identification is lighter but answers a different question. Simple rules are cheap, local and explainable, and are often the first filter a team writes by hand. The question is how much junk such rules actually catch, and how much good text they wrongly flag.

pygarble [7] packages rules of this kind: character and word statistics computed against embedded English tables, with no runtime dependencies. This paper measures it on real data rather than on examples written by its author. **[TODO: one or two sentences on the maintainer's own motivating use case.]**

The contributions are:

- A benchmark on two public datasets: 3,513 survey responses labelled as junk, and 9,971 English rows of random strings and tweets. Every method runs at its own default decision. The protocol, a pilot run and every change after it are disclosed, and per-row predictions are saved so every number can be recomputed.

- A comparison of seven pygarble profiles and six voting modes with six CPU baselines: nostril, a Markov-chain gibberish detector, a chat gibberish detector, a language identifier, and the Gopher and C4 document rules used to filter language-model training data.
- The cost of each method: time per 1,000 texts, import time and install size.
- A plain list of where pygarble is beaten or weak, and guidance on which profile to use for which job.

2 pygarble in one page

pygarble targets Python 3.8 and later and has no runtime dependencies; the installed package, with its embedded English tables, is 1.05 MB. Besides gibberish detection it scans text for secrets, personal data and profanity, with redaction; those features are not evaluated here.

Strategies. Gibberish detection is built from 32 strategies, each a function from text to a score in $[0, 1]$ and a decision. They fall into a few groups: character statistics (vowel ratio, entropy, consonant runs, letter frequency); English character models (a Markov chain, bigram and trigram tables, a log-likelihood ratio against a uniform reference); word-level checks (dictionary lookup, word anomaly, function-word density); structural checks (repetition, keyboard rows and adjacency, hex strings); encoding checks (mojibake, control characters, Unicode script mixing); and four checks that compare a text window with frequent English (cross parsing, primed compression, n-gram rank and a permutation test). Each strategy reports when it does not apply, for example to text with no Latin letters.

Ensembles and profiles. `EnsembleDetector` combines a named set of strategies, a *profile*, under a voting rule. The voting rules are `any`, `majority`, `all`, `average`, `weighted` and `fisher`; a text is flagged when the combined score reaches the threshold, 0.5 by default. The `fisher` rule reads each member's score against that member's scores on a synthetic English null built from word frequencies, and combines the tail probabilities with Fisher's method. The profiles are:

- `english` (the default): Markov chain, log-likelihood ratio, word anomaly, mojibake, keyboard adjacency and control characters, with `any` voting.
- `english_extended`: `english` plus pattern matching, local anomaly and repetition; more aggressive.
- `legacy`: Markov chain, log-likelihood ratio and word anomaly, the members of the previous release's default.
- `english_fusion` (opt-in): word lookup, log-likelihood ratio and cross parsing with `fisher` voting.
- `corruption`: mojibake and control characters only, for encoding damage, independent of English plausibility.
- `spoofing`: a Unicode script and confusable-character check.
- `llm_output`: repetition, control characters, mojibake and local anomaly, for degenerate model output.

`corruption`, `spoofing` and `llm_output` target other kinds of junk than the two datasets contain. They are measured anyway, because a user may reach for them.

What it is not. The English profiles score English plausibility. They are not language identifiers: meaningful text in other languages written in Latin letters can be flagged, and text in other scripts gives them no signal. Scores are heuristic, not calibrated probabilities. A `calibrate()` helper picks a threshold from a user's own labelled examples; this paper does not use it.

3 Datasets and protocol

A: survey junk. The Kaggle dataset `johnwdata/gibberish-text-classification` [3] has 3,767 survey responses, all labelled as junk. Despite its `.xls` extension, the file is a Windows-1252 CSV. We strip each response, drop empty ones and drop exact duplicates, leaving 3,513 rows. A has no clean rows, so the only metric is the *capture rate*: the share of rows a method flags. Its licence is listed as unknown, so the repository stores row indices and hashes, never the text.

B: random strings and tweets. The Hugging Face dataset `Sowmya15/gibberish_multilingual` [9] (Apache-2.0) pools train, validation and test files. We keep the 9,971 non-empty rows whose language is English: 4,000 random strings (junk) and 5,971 tweets (clean). Labels are used as given and duplicates are kept (215 rows repeat an earlier text). We report precision, recall and the false-positive rate (FPR), the share of tweets flagged.

Methods. pygarble 0.11.0 is used with its defaults frozen at commit `e3135ed`; no default or threshold was changed after any result was seen. We run the seven profiles, the six voting modes on the `english` profile, and each of the 32 strategies alone (Appendix A). The `weighted` mode uses equal weights, since none ship, and so matches `average`. The baselines run on CPU with pinned versions:

- **nostril** [2], Hucka's nonsense string evaluator, from GitHub at commit `fbc0c91`; texts it refuses as too short are not flagged.
- **gibberish-detector** 0.1.1 [4], a character Markov model. The package ships no model, so we train one with its own trainer on the corpus its README uses, and flag scores above its command-line default of 4.0.
- **gibberish-chat-detector** 0.1.2 [1], a rule set for chat messages, with its default rules.
- **lingua** 2.2.0 [10], a language identifier, under the policy "not detected as English is junk".
- **Gopher rules** [5] and **C4 rules** [6], the document-quality heuristics used to filter language-model training data, implemented from the RedPajama-V2 quality signal definitions [11] with the original thresholds; the C4 bad-word rule uses the LDNOOBW list [8]. A text is flagged when it fails any rule.

There are no neural baselines.

Decision and intervals. Every method is scored at its own default decision: the binary output it gives with default settings. No threshold is fitted on either dataset. Intervals are 95 % percentile intervals from 1,000 bootstrap resamples of rows, drawn with a fixed seed and shared by all methods.

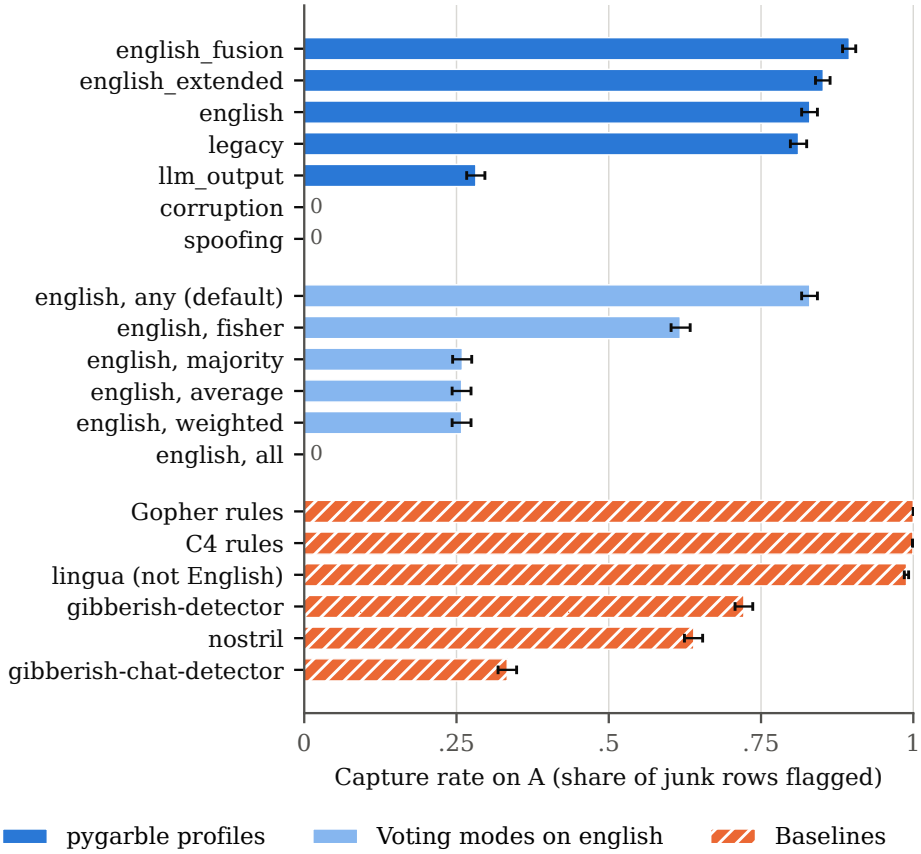


Figure 1: Capture rate on A (3,513 survey responses labelled as junk), every method at its default decision, with 95 % bootstrap intervals. Single strategies are in Appendix A.

Pilot and changes. Before the protocol was written, all of A and all English rows of B were scored once with pygarble's defaults. The `english_fusion` numbers from that pilot are the ones reported here, because nothing in the package changed afterwards. The first version of the protocol also fitted thresholds at fixed false-positive rates on a development split of B; that analysis was removed from scope before this paper was written (Section 9).

4 Results on survey junk

Figure 1 and Table 1 show the capture rate on A. `english_fusion` flags 0.895 of the junk rows (95 % interval 0.884–0.905), the highest of the pygarble profiles. The default `english` profile flags 0.830, `english_extended` 0.852 and `legacy` 0.812.

Three baselines capture more than any pygarble profile: lingua (0.989), and the Gopher (1.000) and C4 (0.999) rules. The rule sets reach these rates by flagging almost everything short, as Section 5 shows. The two dedicated gibberish baselines capture less: gibberish-detector 0.722 and nostril 0.640.

`corruption` and `spoofing` flag nothing in A, as expected: the rows are mashing, not encoding damage or spoofed scripts. Among voting modes, `any` is the profile default; `fisher` captures 0.617; `majority` (0.260), `average` and `weighted` (0.258 each) capture less. `all`, which needs every member to fire, flags nothing.

Table 1: Capture rate on A and precision, recall and FPR on B at each method's default decision, with 95 % bootstrap intervals. Precision is n/a when a method flags nothing.

Method	A capture	B precision	B recall	B FPR
pygarble profiles				
english_fusion	0.895 [0.884, 0.905]	0.906 [0.898, 0.915]	0.991 [0.987, 0.993]	0.068 [0.062, 0.075]
english	0.830 [0.817, 0.843]	0.656 [0.645, 0.670]	0.947 [0.940, 0.954]	0.332 [0.320, 0.344]
english_extended	0.852 [0.839, 0.863]	0.649 [0.638, 0.663]	0.951 [0.944, 0.958]	0.344 [0.331, 0.356]
legacy	0.812 [0.798, 0.825]	0.781 [0.770, 0.793]	0.947 [0.940, 0.954]	0.178 [0.169, 0.188]
llm_output	0.282 [0.267, 0.297]	0.200 [0.184, 0.217]	0.110 [0.100, 0.120]	0.294 [0.282, 0.306]
corruption	0.000 [0.000, 0.000]	0.000 [0.000, 0.000]	0.000 [0.000, 0.000]	0.285 [0.274, 0.296]
spoofing	0.000 [0.000, 0.000]	n/a	0.000 [0.000, 0.000]	0.000 [0.000, 0.000]
Voting modes on english				
english, any (default)	0.830 [0.817, 0.843]	0.656 [0.645, 0.670]	0.947 [0.940, 0.954]	0.332 [0.320, 0.344]
english, majority	0.260 [0.244, 0.275]	0.724 [0.552, 0.880]	0.005 [0.003, 0.008]	0.001 [0.001, 0.002]
english, all	0.000 [0.000, 0.000]	n/a	0.000 [0.000, 0.000]	0.000 [0.000, 0.000]
english, average	0.258 [0.243, 0.274]	0.477 [0.333, 0.630]	0.005 [0.003, 0.008]	0.004 [0.002, 0.005]
english, weighted	0.258 [0.243, 0.274]	0.477 [0.333, 0.630]	0.005 [0.003, 0.008]	0.004 [0.002, 0.005]
english, fisher	0.617 [0.602, 0.634]	0.725 [0.710, 0.741]	0.581 [0.565, 0.595]	0.147 [0.139, 0.156]
Baselines				
gibberish-detector	0.722 [0.707, 0.736]	0.995 [0.993, 0.997]	0.822 [0.809, 0.833]	0.003 [0.002, 0.004]
nostril	0.640 [0.624, 0.654]	0.975 [0.968, 0.981]	0.550 [0.534, 0.566]	0.010 [0.007, 0.012]
gibberish-chat-detector	0.333 [0.318, 0.349]	0.030 [0.021, 0.040]	0.009 [0.007, 0.013]	0.203 [0.194, 0.214]
lingua (not English)	0.989 [0.985, 0.992]	0.876 [0.867, 0.886]	0.994 [0.991, 0.996]	0.094 [0.087, 0.101]
Gopher rules	1.000 [1.000, 1.000]	0.401 [0.392, 0.412]	1.000 [1.000, 1.000]	1.000 [1.000, 1.000]
C4 rules	0.999 [0.999, 1.000]	0.402 [0.392, 0.412]	1.000 [1.000, 1.000]	0.998 [0.997, 0.999]

5 Results on social-media text

Figure 2 shows precision and recall on B. Recall is high for most methods, because B's junk is random strings that share almost nothing with English. `english_fusion` recalls 0.991 with precision 0.906; it flags 6.8 % of tweets.

The precision differences come from tweets. gibberish-detector flags 0.3 % of tweets (precision 0.995) and nostril 1.0 %, both fewer than fusion; their recall is lower (0.822 and 0.550). lingua flags 9.4 % of tweets. The `english` profile flags 33.2 %, `english_extended` 34.4 % and `legacy` 17.8 %. The Gopher and C4 rules flag 100.0 % and 99.8 % of tweets: their word-count and line-structure rules were written for web documents, and almost no tweet passes them.

6 Cost

Figure 3 plots time against capture rate, and Table 2 lists time per 1,000 texts, import time and install size, measured on one thread of an Apple M2 with Python 3.12.2 over all B texts (median of five runs after a warm-up). `english_fusion` takes 60.5 ms per 1,000 texts and pygarble installs in 1.05 MB. gibberish-detector is faster (6.6 ms) at 6.51 MB, including the corpus its model is trained from. lingua is the slowest (1036.4 ms) and largest (307.47 MB). All methods run offline once installed. Timings depend on the machine; the ordering is what carries over.

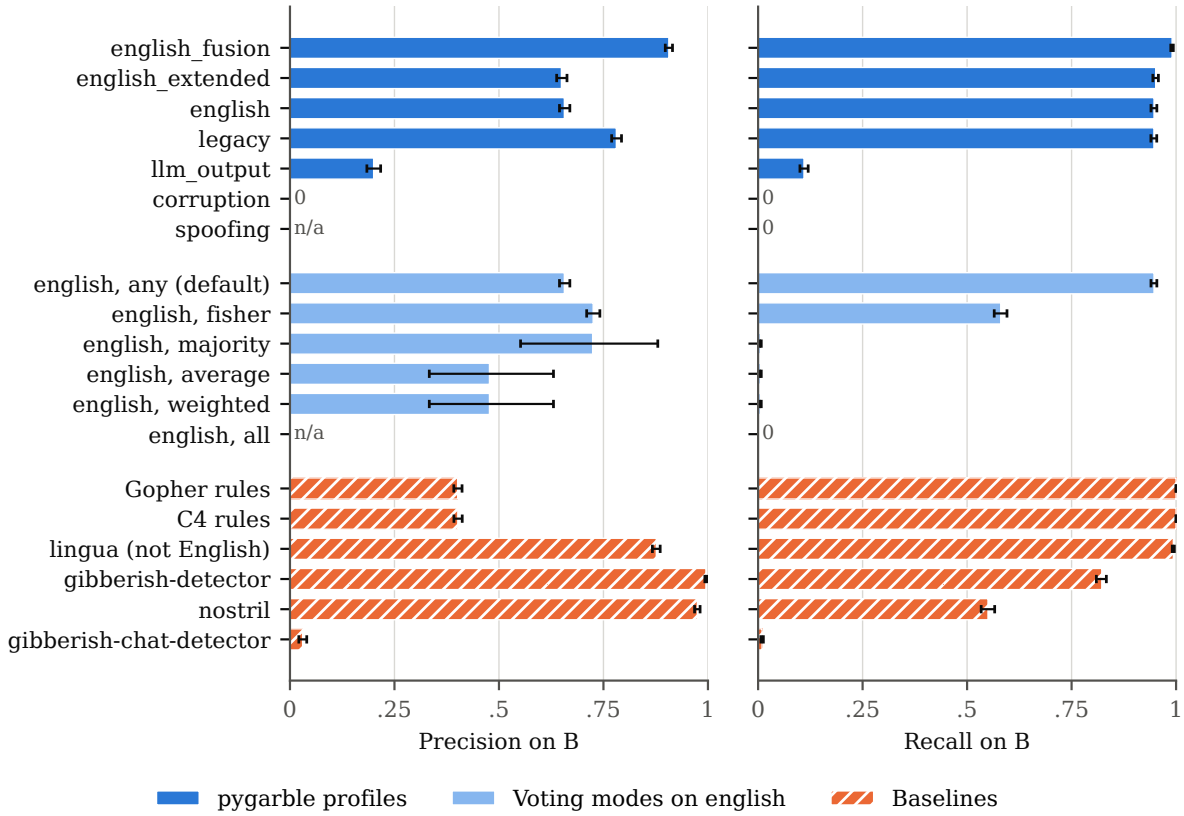


Figure 2: Precision and recall on B (4,000 random strings, 5,971 tweets), every method at its default decision, with 95 % bootstrap intervals.

7 Where it fails

This section lists every place where pygarble is beaten or weak.

False positives on informal text. On tweets, the Markov baseline gibberish-detector has an FPR of 0.3 % against `english_fusion`'s 6.8 %, and nostril 1.0 %. Fusion catches more junk on both datasets, but at default settings it is not the low-false-positive choice.

The default profile. pygarble's default `english` profile flags 33.2 % of tweets, and `english_extended` 34.4 %. Under `any` voting a single member firing is enough. Run alone, the control-character member flags 28.4 % of tweets and the Markov member 17.8 % (Appendix A); the first fires on the C1 control characters inside mojibake (below). These profiles should not be used to drop social-media text without calibration on the user's own data.

Language identification catches more survey junk. lingua captures 0.989 of A, more than any pygarble profile, by calling anything it cannot identify as English junk. It pays with 9.4 % of tweets flagged, 307.47 MB on disk and the slowest time in Table 2, and it would flag every non-English answer.

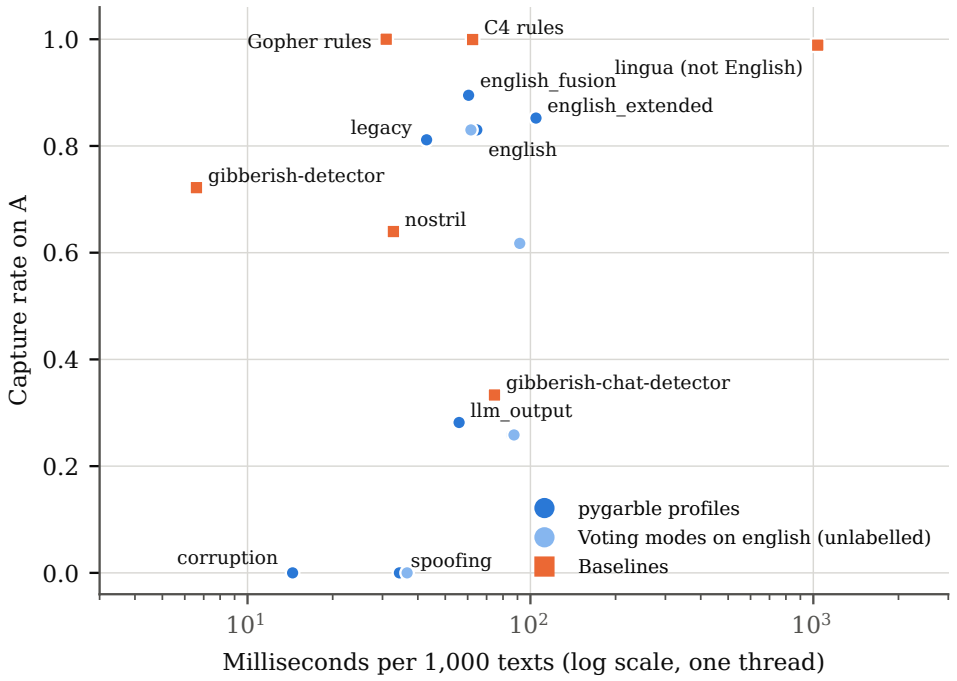


Figure 3: Milliseconds per 1,000 texts (log scale, one thread) against capture rate on A. Voting modes are drawn but not labelled.

Short inputs and word-like mashes. `english_fusion` misses 369 of the 3,513 rows in A. On inspection they are of two kinds. Many are very short: single letters, digits, punctuation or a few repeated letters, which give the English statistics too little evidence. The rest are longer strings built from real words or pronounceable fragments, which read as English to character and word statistics. A minimum-length or character-class rule would catch the first kind; nothing here catches the second reliably.

Profiles built for other junk. `corruption` and `spoofing` flag nothing in A, and `llm_output` flags 0.282. On B, `corruption` flags 28.5 % of tweets, because 26.4 % of the tweets contain mojibake byte signatures (for example a UTF-8 emoji decoded as Windows-1252). Whether those flags are false positives depends on the use: for an encoding audit they are correct, and B labels them clean only because the dataset was built to separate random strings from tweets. The same mojibake contributes to the FPR of the `english` profiles, which include the mojibake check.

Conservative voting modes. At the default threshold, `majority`, `average` and `weighted` voting on the `english` profile recall at most 0.005 of B's random strings, and `all` flags nothing. These modes need a lower threshold, chosen on the user's data, before they are useful.

Label noise in A. A is used as labelled. 15 rows are flagged by none of the four English profiles, nostril, gibberish-detector or lingua. On inspection many are meaningful but low-effort answers, such as thanks or a short positive phrase, and the rest are short mixes of words and fragments. A survey team may want to discard the low-effort answers, but a gibberish detector should not flag them, so A's capture rates have a ceiling below one.

Table 2: Footprint. Time is the default decision over all 9,971 B texts on one thread, median of five runs. Install size counts the installed files of the distribution and its runtime dependencies, plus model or word-list files the method needs.

Method	ms per 1,000 texts	Import (s)	Install (MB)
pygarble profiles			
english_fusion	60.5	0.020	1.05
english	64.6	0.020	1.05
english_extended	104.6	0.020	1.05
legacy	43.0	0.020	1.05
llm_output	56.0	0.020	1.05
corruption	14.4	0.020	1.05
spoofing	34.4	0.020	1.05
Voting modes on english			
english, any (default)	61.6	0.020	1.05
english, majority	87.4	0.020	1.05
english, all	36.7	0.020	1.05
english, average	86.0	0.020	1.05
english, weighted	87.5	0.020	1.05
english, fisher	91.7	0.020	1.05
Baselines			
gibberish-detector	6.6	0.039	6.51
nostril	32.8	0.395	13.25
gibberish-chat-detector	74.6	0.036	20.63
lingua (not English)	1036.4	0.001	307.47
Gopher rules	30.9	0.003	0.01
C4 rules	62.5	0.003	0.01

B is easy for dictionary methods. B's junk is random character strings. A plain dictionary lookup separates them from tweets almost perfectly: the `word_lookup` strategy alone recalls 1.000 of them with an FPR of 4.3 % (Appendix A). B therefore says little about recall on realistic junk and mostly measures false positives on informal English.

8 Guidance

- **Screening survey or form free text for review:** `english_fusion`. It catches 0.895 of the junk answers in A at 60.5 ms per 1,000 texts with no dependencies. Add a minimum-length rule for one- to five-character answers.
- **Dropping rows automatically, where false positives cost more than misses:** at default settings, gibberish-detector's Markov model has the lowest FPR on tweets. Alternatively, calibrate an `english_fusion` threshold with `calibrate()` on a few hundred labelled rows of your own data; this paper does not measure that option.
- **Informal or social-media text:** avoid `english`, `english_extended` and `legacy` at default settings.
- **Encoding damage:** `corruption`, knowing that it will flag mojibake that a human reader would call readable.

- **Multilingual input:** none of the English profiles. Route by language first; a language identifier is the right tool for that question, not for junk detection.
- **Short texts:** not the Gopher or C4 rules, which were designed for web documents and flag almost every tweet.

9 Limitations

Two public datasets are measured, both English. A has only junk, so it measures capture and nothing about false positives; B's junk is random strings, which are easier than real junk. Both datasets are short texts; pygarble's behaviour on long documents is not measured here. There is no neural baseline. A comparison at a matched false-positive rate was computed in PR 18 and removed from scope at the maintainer's direction; it remains available in the repository history. Each method is run once at its default decision, so the results say how the methods behave out of the box, not how well they can be tuned.

10 Reproducibility

The benchmark code is in `paper/scripts/benchmark/`, the protocol and every deviation from it in `paper/benchmark/`, and results in `paper/benchmark/results/`. The manifest records the sha256 hash of each raw file (for A, beginning `091d5d3895f035d5`) and every row id; no dataset text is committed. Per-row scores and decisions for every method are committed, so every number in this paper can be recomputed without the raw data:

```
make benchmark-verify PYTHON=<venv>/bin/python
make preprint PYTHON=<venv>/bin/python
```

The first command recomputes `metrics.json` from the saved predictions and fails on any difference. The second regenerates the figures, the tables and the number macros this paper is typeset from, then builds the PDF. Re-scoring from the raw files is described in `paper/scripts/benchmark/README.md`; scores are deterministic.

AI assistance

Claude Code agents running Claude Opus 5.5, Claude Sonnet 5.5 and Claude Fable 5.1 wrote the benchmark code, ran the experiments and drafted this text, under the author's direction and review. Commit trailers in the repository record the models used.

References

[1] Jiangang Hao. gibberish-chat-detector. `https://github.com/jgbrainstorm/gibberish-chat-detector`, 2026. Version 0.1.2 on PyPI; accessed 2026-10-11.

[2] Michael Hucka. Nostril: A nonsense string evaluator written in Python. *Journal of Open Source Software*, 3(25):596, 2018. doi: 10.21105/joss.00596.

[3] johnwdata. Gibberish text classification. Kaggle dataset, `https://www.kaggle.com/datasets/johnwdata/gibberish-text-classification`, 2026. File `Gibberish.xls`; licence listed as Unknown; accessed 2026-10-11.

[4] Aaron Loo. gibberish-detector. https://github.com/domanchi/gibberish-detector, 2026. Version 0.1.1 on PyPI; a port of https://github.com/rrenaud/Gibberish-Detector; accessed 2026-10-11.

[5] Jack W. Rae, Sebastian Borgeaud, Trevor Cai, Katie Millican, Jordan Hoffmann, Francis Song, John Aslanides, Sarah Henderson, Roman Ring, Susannah Young, et al. Scaling language models: Methods, analysis & insights from training Gopher. *arXiv preprint arXiv:2112.11446*, 2021.

[6] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140):1–67, 2020.

[7] Ujjwal Rao. pygarble: local screening of text for secrets, PII, profanity and gibberish. https://github.com/brightertiger/pygarble, 2026. Version 0.11.0, commit e3135ed.

[8] Shutterstock. List of dirty, naughty, obscene, and otherwise bad words. https://github.com/LDNOOBW/List-of-Dirty-Naughty-Obscene-and-Otherwise-Bad-Words, 2026. English list at commit 5faf2ba; accessed 2026-10-11.

[9] Sowmya15. gibberish_multilingual. Hugging Face dataset, https://huggingface.co/datasets/Sowmya15/gibberish_multilingual, 2026. Apache-2.0; accessed 2026-10-11.

[10] Peter M. Stahl. Lingua: the most accurate natural language detection library for Python. https://github.com/pemistahl/lingua-py, 2026. Package lingua-language-detector, version 2.2.0; accessed 2026-10-11.

[11] Together Computer. RedPajama-Data: quality signals for RedPajama-V2. https://github.com/togethercomputer/RedPajama-Data, 2023. Signal definitions used at commit 6d2cee9.

A Single strategies

Table 3 shows each strategy run alone at its default decision, sorted by capture rate on A. Intervals are in paper/benchmark/results/results.md.

Table 3: Single strategies at their default decisions.

Strategy	A capture	B precision	B recall	B FPR	ms per 1,000
word_lookup	0.941	0.940	1.000	0.043	7.4
markov_chain	0.801	0.781	0.947	0.178	30.9
ngram_frequency	0.769	0.925	0.861	0.047	32.4
pronounceability	0.649	0.912	0.745	0.048	32.9
log_likelihood_ratio	0.624	0.999	0.725	0.000	36.4
vowel_ratio	0.593	0.952	0.388	0.013	27.8
word_anomaly	0.579	0.995	0.580	0.002	30.7
pattern_matching	0.573	0.930	0.476	0.024	45.3
cross_parsing	0.569	0.933	0.560	0.027	37.5
ngram_rank	0.565	0.938	0.490	0.022	90.8
keyboard_pattern	0.367	0.131	0.008	0.036	32.8
consonant_sequence	0.365	0.984	0.190	0.002	7.9
permutation_test	0.345	0.914	0.178	0.011	152.3
keyboard_adjacency	0.320	0.388	0.006	0.007	12.9
primed_compression	0.270	0.970	0.404	0.008	65.5
entropy_based	0.259	0.986	0.054	0.001	24.2
local_anomaly	0.207	0.989	0.110	0.001	31.6
letter_position	0.191	0.544	0.022	0.012	29.1
letter_frequency	0.179	0.000	0.000	0.010	29.6
repetition	0.119	0.000	0.000	0.015	17.1
bigram_probability	0.092	1.000	0.153	0.000	13.5
rare_trigram	0.020	1.000	0.019	0.000	9.5
function_word_density	0.019	0.000	0.000	0.029	6.1
vowel_pattern	0.012	0.105	0.001	0.006	7.1
word_collocation	0.008	0.000	0.000	0.039	8.2
hex_string	0.008	0.000	0.000	0.008	6.8
symbol_ratio	0.002	0.000	0.000	0.002	4.6
affix_detection	0.002	n/a	0.000	0.000	22.5
control_characters	0.000	0.000	0.000	0.284	8.3
mojibake	0.000	0.000	0.000	0.094	7.3
unicode_script	0.000	n/a	0.000	0.000	34.2
zipf_conformity	0.000	n/a	0.000	0.000	3.2